`callable` data is *invokable*; it can be executed. Functions in PHP are *first class* (like in other languages such as JavaScript), which means they can be handled just like any other data, in that variables can reference them, and they can be passed into (and returned by) functions. Chapter 3, "Functions," and Chapter 6, "Object Fundamentals," explain these concepts in more detail.

Finally, the special case: a `resource` is an old concept in PHP that denotes a variable that refers to an external resource. A typical PHP application might read or write to files, send data to an FTP server, interact with databases, and so on. In previous versions of PHP file descriptors, network sockets, database handles, and so on were represented by variables with the data type of `resource`. Many resource types are now being phased out in PHP 8 and replaced by dedicated classes.

Although it isn't possible to declare a variable's type when creating it, it is possible to place restrictions on the types that a function (or object method) will accept as input and/or return as output. The type declaration system in PHP greatly expands on the base types presented here; full details are given in Chapter 3, "Functions."

Operations and Calls

Programs are written to manipulate data and transform it into information. The most fundamental form of data manipulation is an *operation*, that is, the use of *operators* to change, copy, or compare data values. An operation can look like this: `x = 1 + 1` (which adds the values `1` and `1`, then assigns the result to the symbol `x`), or this: `x == y` (compares the values of `x` and `y` and evaluates to a boolean `true` or `false`). There are many built-in operators in PHP; we'll examine them in their respective chapters (arithmetic operators are covered in Chapter 7, "Numbers," string operators in Chapter 8, "Strings," and so on).

A *call* occurs when control is passed to a subprogram within your main program, usually via a function or object method. Often this is done with data being sent to the subprogram from the calling code, and new data is usually returned when the function has completed its work. Calls look like this: `someFunction($var);` – the callable unit is `someFunction`, the parentheses indicate it should be executed, and the variable `$var` is being given to the call so it is available for use in the callable code. It is generally good practice to restrict the types of data the call will accept and declare the type of the return data. PHP's built-in functions and methods are normally type restricted for both incoming data (the arguments) and return data. It is also possible to define input and return types for user-defined calls too.